

我想用react写成html。因为我觉得那个动效最好，是需要多个jsx文件吗。改用react试试吧，每一页也不一定是完全按照现有的内容来，可以更灵活的调整，但尽量最大化利用好600多万条乐评，这些评论的信息挺多，如果你还有很让人惊喜的设计和统计角度也可以提出来我分析好后再拿到新的页展示。

我想用react重写了。来吧!!!每页焕新构思，完整prompt



好，我先理清你要什么，再开干。

你的需求拆开来是：

1. **用 React 重写**，结构上参考第一份（多 jsx 文件分场景），但内容是 mox 那套数据
2. **每页焕新构思**——不被现有 17 页捆死，可增可删可换角度
3. **最大化用好 600 万+乐评**——数据是金矿，要挖得更狠
4. **要求我提出新的统计角度**，你来判断能不能落地

那我先回答两个工程问题，再给你焕新版**的整体页面构思**，最后给完整的逐页 prompt。

先说工程：是不是要多 jsx 文件？

是，但不是物理上的多文件，是逻辑上的多模块。第一份那种 `scenes-1-3.jsx` `scenes-4-6.jsx` 的拆法是个**取巧**——它走的是 `<script type="text/babel">` 浏览器内编译路线，所以才需要拆文件减小单文件压力。

我给你的最终交付建议是：**单 HTML 文件，内嵌 React + Babel standalone**，但代码内部按场景模块化组织（每个场景一个组件 + 一个 hooks 文件 + 一个数据切片）。这样：

- 你拿到就一个 `.html`，双击能跑，无需构建
- 但代码内部清晰分层，后续改某一页不影响别的
- 数据 600 万条不会真的塞进 HTML——会预聚合成几十 KB 的统计 JSON 内嵌

也就是说，最终交付一个 `index.html`，但你打开它能看到清晰的：

```
DATA = { ... }           // 预聚合数据
useScene()                // 场景调度 hook
useAudio()                // 音频 hook
<Scene01Cover />          // 每个场景独立组件
<Scene02Overview />
...
<App />                   // 调度器
```

600 万条评论里还能挖出什么——我提的新统计角度

mox 现有 17 页其实只用到了**最浅的一层**：评论量、长度、情绪、关键词、人称。还有**更让人心头一颤**的角度它没碰：

🔥 让人惊喜的角度（请你判断哪些数据能拉出来）

A. 凌晨评论档案 —— 把所有评论按发布**小时**重新切片。

假说：薛之谦的歌，凌晨 1-4 点的评论占比异常高于其他时段。这群“夜猫子评论者”留下的话，和白天的话**情绪结构完全不同**——他们更长、更私人、更悲伤。这一页可以做成“凌晨三点的评论区”——一面会随真实时间变化的钟，钟指向不同时刻就显示该时刻最高赞的评论。

B. 评论的“潜伏期” —— 某条评论是在歌发行**多久之后**写下的。

把每首歌做成一根时间柱，柱子上标记每条高赞评论的发布距离首发的“年差”。你会发现《认真的雪》最高赞的几条都是 **10-15 年后**写的。这一页叫“**这首歌等了多久才被听懂**”。

C. 反复出现的“日期锚点” —— 提取评论里的具体日期（“2018 年 3 月 14 日”、“高三那年冬天”）。

这些是被记住的“私人纪念日”。可以堆成一张**生日卡墙**——每条评论变成一张写有具体日期的卡片，集体展示“那么多人，记得那么多不同的日子”。

D. 同句话的“重复发现” —— 在不同歌的评论区里，**相同或高度相似的短句**反复出现的次数。

比如“我没事”、“原来这首歌写的是我”、“上一秒还在笑”——这些套话是评论区的**集体修辞**。做一张“**评论区共用的句子**”图，列出 Top 30 的“被反复写下的句子”。这比关键词云有人味。

E. 评论里的“被点名” —— 用户在评论里 @ 别人或提到具体人名（“@张三 还记得吗”、“想起了 XXX”）的密度。

评论区不只是对薛之谦说话，**也是对评论里另一个人说话**。这个洞察很温柔。可以做成“**这条评论是写给谁的**”——把高赞被 @ 评论拎出来。

F. 编辑过的评论 —— 网易云有些评论会显示“已修改”标记（如果数据里有这个字段）。

哪些评论被作者反复回去修改？为什么？可以做“**反复回头修改的那些话**”——这是一种特殊的私人写作。

G. 评论的“长度分布双峰” —— 长度直方图很可能是双峰的（短评 5-15 字，长评 80-200 字）。

如果是这样，说明评论区其实是**两个完全不同的群体**在共存：路过的和驻扎的。做成两座并立的山。

H. 高赞评论的"获赞速度"——一条高赞评论是**发出后多久**积累到 1 万赞的。

有的几小时爆，有的写出来三年没人理，第七年突然被顶。这个"延迟成名"现象单独一页，叫"评论的来生"。

I. 跨歌"主题接龙"——同一种情绪关键词（比如"失眠"）在哪些歌里被高频提及，按年份连成一条线。

"失眠"在 2008 年是《认真的雪》评论区的高频词，2017 年迁移到《丑八怪》，2022 年迁移到《尘》。一种情绪，**找了二十年宿主**。

J. 评论被引用为别人评论的"引用网络"——长评里引用别人评论或歌词的比例。

评论区的话**互相引用**形成网。一条评论被几条评论引用过，构成"评论圈层"。

K. 评论里的"地点"——提取评论里的地名（"在重庆"、"上海地铁"、"成都的雨"）。

可以做一张中国地图，地名密度铺成热力——**这首歌在哪些城市被反复听**。极有诗意。

你来回答我：上面 A-K 哪些数据你拉得到？（地名 K、@ 被点名 E、修改字段 F 这几个不一定有；A B C D G H I J 应该都能从原始评论文本里 derive 出来）

我的优先级建议：A（凌晨）、B（潜伏期）、C（日期锚点）、D（共用句子）、I（情绪宿主迁移）——这五个最容易做出**情感冲击**而不只是好看。

焕新页面构思（在 mox 17 页基础上重组）

假设 A B C D I 五个新角度都能拉出数据，我给你一个**14 页焕新结构**——比 mox 少 3 页（删掉重复的 P4 和 P14，合并 P10 P11），加进新角度：

#	标题	主导动作	数据来源
01	声音先于字	听 + 看封面浮现	静态
02	187 万人写过的话	看一个数字滚出来	总量统计
03	45 首歌的星图	扫光巡游 + 手电筒	歌曲基础数据
04	这首歌等了多久才被听懂（新·B）	看潜伏期柱子展开	评论时差
05	凌晨三点的评论区（新·A）	拨动一只钟	评论时段切片
06	被反复写下的句子（新·D）	句子从墙上"飘下来"	相似句聚类
07	评论区在贴便签	拖动便签墙	高赞评论文本

#	标题	主导动作	数据来源
08	二十年的语言搬家	词云三段渐变	高频词分时期
09	这只是一条长评	一条评论被一字字读出	长评样本
10	情绪在搬家	粒子迁移	情绪时序
11	一种感觉，找了二十年宿主（新·I）	一个词在歌之间跳跃	关键词跨歌追踪
12	被记住的那些日子（新·C）	日期卡墙	评论中的日期提取
13	尘	听一首歌	音频 + 频谱
14	世界和平	18 万条同句汇聚	"世界和平"全检索

完整逐页 React 实现 prompt（给你或给下一个 AI 的施工图）

下面每一页都是一个独立的 React 组件 spec，含：组件名 / props / 内部状态 / 视觉规格 / 动效时间线 / 交互 / 文案。你可以把任何一段单独丢给 AI 让它实现某一页。

全局 setup

单文件 index.html, 内嵌:

- React 18.3.1 + ReactDOM (UMD, dev build)
- Babel standalone 7.29
- Tailwind 不要 (避免 CDN 编译开销), 用纯 CSS variables + scoped styles
- Google Fonts: Noto Serif SC (700/900), Noto Sans SC (300/400/500), JetBrains Mono (300/500), Ma Shan Zheng

CSS 变量保持 mox 的色板:

```
--bg:#07050d; --ink:#f5f1e6; --muted:#9b93a8;  
--pink:#ff557f; --orange:#ff9966; --gold:#ffd06b;  
--cyan:#73d6ff; --violet:#aa95ff; --green:#7dd19d;  
--paper:#f4ede1; --paper-soft:#e0d7c8;
```

字体语义:

```
--serif: 衬线大字 / 标题 / 评论正文  
--sans: UI / 数据  
--mono: chrome / 元数据 / 技术信息  
--hand: 手写感强调 (Ma Shan Zheng)
```

全局组件:

```
<App> 调度场景索引 sceneIdx, 暴露 next/prev/jumpTo  
<Stage scenes={[...]}> 渲染当前 scene + 前后 1 帧 (提前挂载, opacity 控制可见)  
<Chrome corner="tl|tr|bl|br"> 角落小字  
<ProgressBar total={14} active={i} onJump={...}> 细横线 + 段落点  
<NoiseLayer /> 全局噪点 SVG (mox 已有)  
<ScanlinesLayer /> 扫描线 (mox 已有)  
<VignetteLayer /> 暗角 (mox 已有)
```

全局 hooks:

```
useScene()      场景切换 (键盘 ←→, 滚轮节流, 触摸滑动, 进度条点击)  
useAudio()      全局音频 (Dust mp3, 可暂停, 暴露 analyser)  
useEnterAnimation(sceneActive, delay) 场景进入时触发动画  
useReducedMotion() 检测 prefers-reduced-motion, 关掉粒子和持续动画
```

数据形状 (DATA 对象, 预聚合后约 80-150KB):

```
DATA.stats      总量、长评率、超粉、avgLength  
DATA.songs[45]  每首歌: id/title/year/album/comments/topComment/dominantEmotion/sentimentScore/nostalgiaScore  
DATA.hourlyComments[24] 评论时段切片  
DATA.delayPattern[] 每首歌评论的"年差"分布  
DATA.recurringPhrases[] 共用句子 + 出现次数 + 例子  
DATA.dateAnchors[] 评论中提取的具体日期  
DATA.emotionMigration{ early:[], mid:[], late:[], flows:[] }  
DATA.themeJourney[] 关键词跨歌迁移 (每个词一条 timeline)  
DATA.peacePhrases[] "世界和平"类语句样本  
DATA.dust       { audioMp3, audioFlac, cover, year, comments, topComment }
```

Scene 01 · <CoverScene />

意图: 声音先于字。让用户在 4 秒内从"加载完成"过渡到"被吸进去"。

结构:

<CoverScene>	
— <SpectrumLine />	一条横向频谱（单一 path，不是竖条）
— <TitleBurst />	"听过他的人" 五字逐字炸出
— <Subtitle />	斜塞的"认真"二字（手写体）
— <BreathingDot />	"按任意键" 呼吸红点
— <CornerChrome />	四角元数据（鼠标静止 2s 才显现）

动效时间线（进入此场景时）：

- t=0: 全黑屏，正中一条 1px 白线
- t=0.8s: 白线开始按 audio analyser 数据起伏（fakeSpectrum 兜底）
- t=1.5s: "听" 字从频谱中央"震出"，font-size 从 0 → 14vw，0.4s ease-out
- t=1.8s: "过" 字震出，落在"听"右侧（允许 5px 重叠）
- t=2.1s: "他" "的" "人" 每字间隔 0.3s 依次震出
- t=3.2s: "认真" 二字从"听"字左上角小字滑入，30°斜置，手写体
- t=3.6s: 副标题手写两行从下方淡入 "—— 我没办法听完所有人 / 但我把他们写下的话，留下来了"
- t=4.0s: 底部呼吸红点 + "按任意键，让声音先进来" 出现，循环呼吸 2.4s
- 用户按任意键或点击：音频淡入（1.2s），频谱炸成全屏涟漪，切到 P02

关键实现细节：

- 字逐字震出用 keyframes + 逐元素 animation-delay
- 频谱用单 <path>，每帧用 analyser 数据生成 SVG path d 属性（非 canvas，避免分辨率问题）
- useAudio() 暴露 getAnalyserData()，没数据时 fakeSpectrum(t) 兜底
- 角落 chrome 用 setTimeout + mousemove 监听，2s 静止才 fade-in

文案最终版：

- 主标题: 认真听过他的人（"认真"为手写小字斜塞）
- 副标题（手写两行）：

—— 我没办法听完所有人
 但我把他们写下的话，留下来了
- 红点旁: 按任意键，让声音先进来
- 角落: CloudNet · 2026 / 45 SONGS · 6.4M COMMENTS / A DATA STORY / 2006 — 2026

Scene 02 · <NumberRollScene />

意图：让 600 万这个数字自己长出来。整页一件事。

结构：

```

<NumberRollScene>
  |— <RollingBigNumber target={6432819} /> 老虎机式滚动主数字
  |— <FloatingPhrases /> 背景飘评论碎句
  |— <TypewriterCaption /> 数字下方逐字打出说明
  |— <SatelliteStats /> 三个小数字"长评 18% / 超粉 4823 / 平均 47
字"

```

动效时间线:

- t=0: 屏幕只有居中数字 0 (font-size: 22vw, 衬线 900)
- t=0.3s: 数字开始滚动 (每帧随机 0-9 数字, 2.8s 后 easeOutExpo 减速到真值)
- t=0.6s: 背景开始生成"飘字"—每 200ms 生成 1 条真实评论碎句 (6-12 字), 从屏幕外随机方向飘向中心数字, 被"吸"入时 opacity 0
- t=2.8s: 数字最后 0.4s 锁定, 停止滚动
- t=3.4s: 飘字停止生成, 剩余飘字淡出
- t=3.6s: 数字下方打字机出 "这是 45 首歌下面, 6,432,819 个人留下的话。" (速度 60ms/字)
- t=5.2s: 数字下方衬线小字 "评论区不是数据, 是声音的副本。" 淡入
- t=6.0s: 三个小卫星数字横排展开 (各自 0.6s 滚动)

关键实现细节:

- <RollingBigNumber> 用 useState 存当前显示值, requestAnimationFrame 循环; 每帧用伪随机数 Math.floor(Math.random() * Math.pow(10, digits)) 直到最后 12% 时间锁定真值
- <FloatingPhrases> 内部维护 phrases: [{id, text, x, y, vx, vy, life}] 数组, 每 200ms push, 每帧更新位置 + 减 life
- 飘字"被吸入"逻辑: 当距中心数字 < 100px 时, 加快速度趋向中心 + opacity 衰减
- 数字字符等宽用 font-variant-numeric: tabular-nums

文案:

- 大数字下方第一行 (打字机): 这是 45 首歌下面, 6,432,819 个人留下的话。
- 第二行 (衬线): 评论区不是数据, 是声音的副本。
- 卫星数字: 长评 18% 超级粉丝 4,823 人 平均长度 47 字
- 角落: 02 / 14 DATASET OVERVIEW

Scene 03 · <TimelineConstellationScene />

意图: 45 首歌一次性铺出来, 先扫光巡游让所有歌都被看到, 再开放手电筒探索。这是焕新版的核心实验页。

结构:

<code><TimelineConstellationScene></code>	
<code> — <SineTimeAxis points={45} /></code>	正弦波时间轴 + 45 光点
<code> — <ScanLight phase="auto manual" /></code>	自动扫光 / 手电筒模式
<code> — <SongPopup activeSongId={...} /></code>	扫光经过时浮出歌名 + 热评
<code> — <ReadCounter count={n} total={45} /></code>	"已读 N/45"
<code> — <TouchedList songs={[...]} /></code>	用户点击过的歌堆叠
<code> — <ModeToggle mode="auto manual torch" /></code>	底部模式切换

默认进入流程：

- t=0: 屏幕暗，正弦波时间轴用 `stroke-dashoffset` 从左到右画出 (1.2s)
- t=1.2s: 45 个光点按发行年份在波上 fade-in (错开 30ms)
- t=2.0s: 评论量前 5 的光点开始呼吸光晕循环
- t=2.5s: 自动扫光从 2006 端启动，向 2026 端移动，**总时长 18s**
 - 扫光是高斯模糊的椭圆光圈 (180px 宽)，CSS `filter: blur(20px)` + 渐变
 - 经过光点：光点 scale 1.4, 上方歌名衬线大字下落 (持续 0.7s)，下方热评碎句飘出 (持续 1.2s)
 - 经过 Top5 歌时：扫光暂停 0.4s，热评字号加大
 - 已读计数器每经过一首 +1
- t=20.5s: 扫光淡出，底部出"想细看哪一首，自己点。"
- 进入自由 hover/点击模式

手电筒模式 (按 F 或点底部灯泡)：

- 全屏叠加 80% 黑色蒙版 (`<div>` 定位 + `mix-blend-mode: multiply`)
- 鼠标位置生成 220px 圆形透光区 (`radial-gradient mask`)
- 透光区内的光点正常显示，区外完全隐入黑暗
- 鼠标悬停某光点 0.5s：播放该歌音频前 2 秒 (如有)，浮出最高赞评论
- 再按 F 退出

自由模式交互：

- hover 光点：放大 + 浮出热评卡片 (自动避让屏幕边缘，用 `useRef` + `getBoundingClientRect` 判断)
- 点击光点：加入"我读过的"列表 (屏幕右侧堆叠最多 5 张迷你卡片，超过自动滑出)
- 左上角"重放巡游"按钮，点击重置回 t=0

关键实现细节：

- 时间轴 SVG path: `M 0 270 Q 250 230 500 270 T 1000 270` (正弦近似)
- 光点位置: 用 `path.getPointAtLength(t * pathLength)` 计算每首歌在波上的精确坐标
- 扫光位置: 用 `requestAnimationFrame` + 单一 state 推进 0→1 进度

- 手电筒蒙版: `<div style={{ background: 'black', WebkitMaskImage: \ radial-gradient(circle 110px at  $maxpx\{my\}px$ , transparent, black)` }} />`

文案:

- 标题 (页内左上): **45 首歌, 20 年。**
- 副标题: 先让光自己走一遍, 再决定你要在哪首歌停下来。
- 巡游结束: 想细看哪一首, 自己点。
- 模式切换底部小字: 自动巡游 自由探索 🔦 手电筒 (F)
- 计数器: 已读过 N / 45

Scene 04 · `<DormancyScene />` NEW

意图: 每首歌从发行到被高赞评论"找到"花了多少年。这是 mox 没做的角度。

核心数据: 每首歌的最高赞 Top 10 评论的发布距离首发的"年差"。

结构:

```
<DormancyScene>
  |— <DormancyChart>
  |   <SongRow>
  |   ...
  |— <SortToggle by="dormancy|year|comments" />
  |— <FocusCard song={hoveredSong} /> 右侧 hover 详情
  |— <Insight />                        关键洞察文字
```

视觉:

- 横轴: 年份 2006 – 2026
- 每首歌一行, 行高 18px。从屏幕顶部到底部排列
- 每行最左边: 歌名 (小字衬线) + 一个**红色圆点**标记发行年
- 每行后面: **Top 10 高赞评论的散点**, 按发布年份分布在该行上
 - 散点大小 = 该评论赞数 (log)
 - 散点颜色: 发布距首发 **0-1 年内** 用浅蓝 ("当下被听到"), **2-5 年** 用米色, **5+ 年** 用粉色 ("延迟被听到")
- 默认按"潜伏中位数"降序排列——潜伏最久的歌 (《认真的雪》《意外》之类老歌) 在最上面

进入动画:

- t=0: 仅显示年份横轴
- t=0.3s: 每行从左到右"长"出来 (错开 25ms), 先出歌名 + 红点
- t=1.5s: 每行的 Top 10 散点逐个 fade-in + scale 弹入 (错开 15ms)

- t=4s: 屏幕中央叠出文字: "红点是发歌的那一天。粉点是真的有人听懂的那一天。" 停留 3s 后淡出

交互:

- hover 任一行: 该行高亮 (其他行 30% 透明度), 右侧浮出该歌的"延迟最远"那条热评原文
- hover 任一散点: 浮出该评论的发布年份 + 距首发 X 年 + 该评论原文

关键 trick:

- 屏幕底部固定一行摘要文字: "45 首歌里, 有 12 首的最高赞评论, 写于发行 5 年之后。"——这是这页的金句
- 默认排序后用户能立刻看到: "上面这一片粉点, 全是当年没多少人在意的歌"

文案:

- 标题: 这首歌等了多久, 才被听懂。
- 副标题: 红点是发歌那天。每根线后面的散点是它最高赞的几条评论——它们写于哪一年。
- 底部金句: 45 首歌里, 有 12 首的最高赞评论, 写在发行 5 年以后。
- 排序选项: 按潜伏期 / 按发行年 / 按评论量

Scene 05 · <NightCommentsScene /> NEW

意图: 把评论按 24 小时切片。展示"凌晨评论区"是另一个世界。

核心数据: 每小时 (0-23) 的评论占比 + 每小时最高赞评论。

结构:

```
<NightCommentsScene>
  |— <RadialClock hour={selectedHour} onChange={...} /> 中央大表盘
  |— <HourBreakdown hour={...} /> 该时段的指标 + 热评
  |— <DistributionRing /> 表盘外圈: 24 段评论量分布
  |— <NightHighlight /> 凌晨 1-4 点的特别强调
```

视觉:

- 屏幕中央一个**大表盘** (直径 60vh), 表盘指针实时指向当前选中小时
- 表盘外圈是一圈**24 个扇形**, 每个扇形高度 = 该小时评论量占比
 - 凌晨 1-4 点的扇形用**粉色**, 其他时段用米色——视觉上一眼看出"夜间凸起"
- 表盘中央显示当前选中时刻: "03:00" 衬线大字
- 表盘下方: 该时段三条最高赞评论卡片

默认进入:

- t=0: 表盘从 12 点位置开始, **自动转一圈** (5s), 指针扫过每个小时

- 扫过任何小时时，外圈对应扇形高亮闪一下
- 中央时间数字跟着变 00:00 → 01:00 → ... → 23:00
- t=5s: 指针停在 **03:00**，外圈凌晨 1-4 点扇形一起脉动一下
- t=5.5s: 中央数字下方淡出"这是评论区最不睡觉的四个小时"
- t=6s: 下方出现 03:00 时段的三条最高赞评论

交互：

- 用户**拖动表盘指针**（鼠标按下指针 + 移动）→ 实时切换到该小时的内容
- 用户**点击外圈任一扇形** → 指针快速旋转到该小时
- 拖动时表盘有阻尼感（拖到一格自动吸附）

关键洞察展示：

- 屏幕右上角小字（永远在）："凌晨 1-4 点的评论：占总量 19.3%，但平均长度比白天长 64%。"
- 这个数据让"凌晨评论=更深的评论"这个洞察被坐实

关键实现细节：

- 表盘用 SVG 画
- 拖动用 mousedown 在指针上 + mousemove 算 atan2 角度
- 24 个扇形用 SVG path 用三角函数批量生成

文案：

- 标题（页内左上小字）：**凌晨三点的评论区**
- 副标题: 同一首歌，白天和深夜的评论，是两种语言。
- 中央默认时间下方: 这是评论区最不睡觉的四个小时
- 右上数据: 凌晨 1-4 点占总量 19.3%，但平均长度比白天长 64%。

Scene 06 · <RecurringPhrasesScene /> NEW

意图：评论区有一套**集体修辞**——同样的话被几百几千人写下。展示这种"集体无意识"。

核心数据：跨歌出现 N 次以上的相似句子（用句向量聚类或 simhash）。

结构：

<code><RecurringPhrasesScene></code>	
<code>├── <PhraseWall phrases={Top30} /></code>	墙：30 个高频共用句子
<code>├── <PhraseDetail phrase={selected} /></code>	点击后浮出：该句被多少人写过 + 出现在哪些
歌	
<code>└── <DropdownReveal /></code>	句子从墙上"飘下来"动画

视觉：

- 整屏黑色背景
- **30 个共用句子**以不同字号铺满屏幕——字号映射出现频次（最多的那句最大），位置半随机但保证不重叠
- 每条句子用衬线大字，颜色是不同的米色调
- 每条句子下方一行 mono 极小字："× 1,847"（被多少人写过）

默认进入：

- t=0: 屏幕全黑
- t=0.4s: 句子从屏幕顶部一句句"掉下来"——错开 80ms，掉到各自位置时有轻微弹跳（CSS `cubic-bezier(.17, .67, .3, 1.4)`）
- t=4s: 全部就位
- t=5s: 屏幕中央叠出一行字："这些不是同一个人写的。" 停留 2s 后淡出
- t=7s: 再叠一行："是不同的人，在不同的歌下面，写下了同一句话。" 停留 2s 后淡出
- 之后进入交互模式

交互：

- hover 任意句子：该句轻微放大 + 发光，其他句子降到 30% 透明
- 点击任意句子：屏幕中央浮出详情卡片：
 - 该句完整版（如果墙上是截断的）
 - "× 1,847 人写过类似的话"
 - "最早出现于 2009 年《认真的雪》"
 - "最近一次出现于 2025 年《尘》"
 - 该句子在 45 首歌里的分布点阵图——每首歌一个小方块，写过这句的歌方块亮起

关键 trick：

- Top 1 那句很可能是 "我没事。" 或 "原来这首歌写的是我。" 之类——把它放在屏幕正中央最大字号，作为视觉锚点
- 整页的核心情绪是 "孤独的人在一起写同一句话"——文案要扣住这个

文案：

- 标题（页内右上小字）：被反复写下的句子
- 副标题：评论区有它自己的修辞——同一句话，被几百几千个不认识的人，分别写下。
- 中央叠字 1: 这些不是同一个人写的。
- 中央叠字 2: 是不同的人，在不同的歌下面，写下了同一句话。
- 详情卡: × N 人写过类似的话 / 最早 / 最近 / 分布

Scene 07 · <StickyWallScene />

意图：mox 那张"证据墙"焕新版——便签墙。让评论真的被"贴"在墙上。（保留 mox 的方向，做到位）

结构：

```
<StickyWallScene>
  |— <WallBackground />          深色水泥墙背景
  |— <StickyNote × 16 />         便签
  |— <BigStickyTitle />          左上压着的大便签（手写体）
  |— <FocusedNote note={selected} />  点击后中央放大
```

视觉：

- 背景：深灰水泥质感（CSS noise + 暗色径向渐变）
- 16 张便签随机散布，每张：
 - 米色 / 暖白 / 浅黄三色之一随机
 - **轻微旋转 -6° 到 +6°**
 - 阴影投在墙上（box-shadow + 偏移）
 - 左上或右上贴一段**半透明米色"胶带"**（CSS 倾斜矩形）
- 便签内容：
 - 一段评论（15-80 字，刻意混合长度）
 - 底部 mono 小字：歌名 · 年份
- 屏幕**左上角**压着一张更大的便签（约 380×260px），手写体（Ma Shan Zheng）写：

我贴了一些他们说过的话。

不是用来证明什么。

只是怕，自己以后忘了。

默认进入：

- 大便签先 fade-in (0.6s)
- 16 张便签**一张张从屏幕外不同方向"飞入"**——错开 0.15-0.4s，飞入轨迹是抛物线，落地时有"啪"的轻微抖动（scale 1.05 → 1）
- 整个入场约 5s

交互：

- hover 便签：旋转角度 → 0°（"被拨正"），scale 1.04，z-index 拉到最高
- 点击便签：飞到屏幕中央放大 1.6 倍（FLIP 动画），其他便签 blur(8px) + dim
- 中央放大状态再点 / 按 Esc 收回

关键 trick：

- 进入这页时**有一个轻微的"按图钉"音效**（每张便签落地时播放一声，可选）——mox 整份没声音，加这一处太点睛
- 便签胶带颜色随机变化：浅黄 / 浅米 / 浅灰，让画面像真的有人粘的

文案：

- 大便签（手写体 3 行）：我贴了一些他们说过的话。/ 不是用来证明什么。/ 只是怕，自己以后忘了。
- 角落：07 / 14 EVIDENCE WALL

Scene 08 · <LanguageMigrationScene />

（合并了 mox 原 P4 P8——这俩本来就是一个意思）

意图：早/中/近三段的语言重心移动。

结构：

```
<LanguageMigrationScene>
|— <ThreeEraColumns>          三段词云
|   <FloatingWordCloud era="early" />
|   <FloatingWordCloud era="mid" />
|   <FloatingWordCloud era="late" />
|— <BigYearLabels />          三个超大年份标签
|— <CommentTicker />          底部飘评论 ticker
|— <RepresentativeWord />      三段顶部各自的"代表词"
```

视觉：

- 整屏分横向三段（左 / 中 / 右），无分割线
- 每段顶部：超大年份范围（衬线 900，深灰）
 - 2006 — 2012
 - 2013 — 2018
 - 2019 — 2026
- 每段年份下：一个真的从数据选出来的"代表词"（衬线 900，金色）
 - 早期: **作品**（中字号）
 - 中期: **故事**（大字号）
 - 近期: **我**（最大字号——视觉上压制其他两段）
- 每段下方：一团高频词漂浮，字号映射频次，xy 位置半随机
 - 词缓慢漂浮（CSS keyframes 各自不同 delay/duration，幅度 ±10px）
 - 三段词云的边界**互相略有重叠**——隐喻"语言不是突然变的"

默认进入：

- t=0: 屏幕只有三个年份大字
- t=0.5s: 三个代表词 fade-in
- t=1s: 词从屏幕左下方一个点喷涌而出，飞向各自位置——早期的词最先到位（错开 30ms），近期的词最后
- t=4s: 全部就位，进入漂浮循环

交互：

- hover 任一词：该词放大 1.3 + 发光，屏幕底部 **ticker 区域** 飘出含该词的真实热评（4s 后淡出）
- ticker 是固定区域，从右到左飘字
- 用户可以快速 hover 多个词，ticker 排队播放

关键 trick：

- 词的位置**不要均匀网格**。要呈现"团块感"——意思相近的词聚集，对立的词远离。这要么手动设定 xy，要么实现一个简单的力导向。
- 顶部三个代表词的字号差**视觉上压制**其他两段——观众不需要看数字，眼睛已经感觉到"近期'我'变得很大"

文案：

- 标题（屏幕底部小字）：**早期的人在说歌；近期的人在说自己。**
- 顶部小字：鼠标停在词上，听这个词背后的人说了什么。
- 角落：08 / 14 LANGUAGE SHIFT

Scene 09 · <LongCommentScene />

（mox 原 P7 焕新——让一条长评被"读出"）

意图：让观众真的读完一条长评。比看 metric 数字有冲击 100 倍。

结构：

<LongCommentScene>	
├── <ScrollingLongTextWall />	背景滚动文字墙
├── <CenterLongCard>	中央大卡片
│ <TypewriterText />	逐字显现
│ <HandwritingSignature />	手写体署名
│ <CardActions />	"换一条" / "看数据" 按钮
└── <FlipBackside />	卡片翻面后的数据

视觉：

- 背景：很多长评以 8-12% 透明度铺满整屏，密密麻麻像报纸版面
 - 字号 11px，行距紧凑

- 整张文字墙缓慢从下往上滚（每秒 8px）
- 屏幕**正中央**叠一张大卡片：
 - 米色厚纸质感（#f4ede1 + 微凸阴影 + 内边距 50px）
 - 宽度约 55vw，高度自适应
 - 内部一段长评（约 200-300 字），衬线，字号 18px，行高 1.85
- 长评下方：手写体署名 "—— 某用户，2019 年 11 月，《最好》评论区"

默认进入：

- 背景文字墙立即出现，开始滚动
- 卡片 fade-in (0.6s)
- 卡片内长评**逐字浮现**（每字 35ms）——读完约 7s
- 读完后下方手写体署名 fade-in
- 再过 1s，卡片右上角出两个小按钮：换一条 看数据

交互：

- 点击"换一条"：当前长评 fade-out，从样本库切到下一条，重复浮字流程
- 点击"看数据"：卡片**翻面**（CSS 3D rotateY，0.8s）
 - 背面是数据：长评只占评论 18%，平均赞是短评的 N 倍，长评分布
 - 背面也用纸张质感，但用列表 + 小图
- 再点击翻面按钮翻回

关键 trick：

- **没有顶部标题**——这页的标题就是长评本身
- 长评读完后，卡片下方淡出一行手写小字："写得长，是因为短的话装不下。"

文案：

- 卡片署名：—— 某用户，2019 年 11 月，《最好》评论区
- 卡片下方手写：写得长，是因为短的话装不下。
- 翻面后小标题：关于"长评"这件事
- 翻面后金句：这只是 N 万条长评里的一条。

Scene 10 · <EmotionParticleFlowScene />

意图：mox 原 P11 焕新。把 Sankey 改成持续粒子流。

结构：

<code><EmotionParticleFlowScene></code>	
<code> — <ThreePhaseColumns></code>	三个垂直柱（早/中/近）
<code> — <ParticleSystem /></code>	canvas 粒子系统
<code> — <SelectedPath path={...} /></code>	点击后只剩选中路径
<code> — <PhaseReadingPanel /></code>	右侧动态文字

视觉：

- 三个垂直柱（屏幕 25% / 50% / 75% 横向位置），柱高约 60vh
- 每柱内部按 5 种情绪从上到下分段（颜色对应 max 现有色板）
- 段高度 = 该情绪在该阶段占比
- 段标签写在段右侧（mono 小字 + 数字）

粒子系统（核心）：

- 用 `<canvas>` 在三柱之间的空白区域绘制粒子
- 每秒生成 30-50 个粒子，每个粒子：
 - 出生位置：早期柱某一段内随机点
 - 颜色：该段对应的情绪色
 - 终点：根据真实迁移权重，按概率选择中期柱的某一段
 - 路径：贝塞尔曲线（控制点带轻微随机）
 - 速度：约 2-3s 跨完一段
 - 到达中期后自动继续迁移到近期，然后销毁
- 流量大的迁移路径粒子密度自然就高

默认进入：

- t=0: 三柱出现，但内部段是空的灰色
- t=0.3s: 每柱从下往上填充颜色（对应每段 height）
- t=1s: 第一段（喜欢/支持）开始放粒子，0.6s 后第二段加入，依次
- t=4s: 五种粒子全部在三柱间穿梭，画面达到稳态
- 一直循环

交互：

- 点击早期柱的任意一段：
 - 其他段全部 dim 到 30%
 - 只剩这段发出的粒子继续流动
 - 右侧文字面板更新为该段的"流向解读"
- 再点空白处恢复全部

右侧文字面板（动态）：

默认状态：
"五种情绪，二十年。
看着它们慢慢搬家。"

选中早期·悲伤后：
"早期的悲伤
→ 中期·怀旧/释怀（68%）
→ 近期·释怀/成长（54%）"

他们说唱的是青春的尾巴。
后来青春过去了，
这种悲伤也长大了。"

关键 trick：

- 粒子系统用 requestAnimationFrame + canvas
- 每粒子带 birth, lifespan, srcSeg, dstSeg, controlPoints 属性
- 粒子总数控制在 200-400，避免性能问题

文案：

- 标题: 情绪在搬家。
- 副标题: 二十年里，一种感觉是怎么变成另一种感觉的。

Scene 11 · <ThemeJourneyScene /> NEW

意图：一个关键词（比如"失眠"）在二十年里寄居于不同的歌。让用户看到一种情绪如何"找宿主"。

核心数据：选定 5-8 个标志性关键词，每个词在 45 首歌评论区的出现频次和年份分布。

结构：

```
<ThemeJourneyScene>
  |— <KeywordPicker words={['失眠','后悔','长大','想她','原谅','一个人','回不去','再见']] />
  |— <JourneyMap />
  |— <NarrativeText />
```

主图：歌曲排成时间轴 + 词的"路径"
当前选中词的故事化叙述

视觉：

- 屏幕上方一排**关键词标签**——8 个词作为按钮，可点击切换
- 屏幕主体：45 首歌按发行年份**横向排列**为 45 个圆点（小，灰色），下方写歌名小字
- 选中某个词后：
 - 该词在哪些歌的评论区高频出现，那些歌的圆点**亮起**（颜色对应词的属性）
 - 圆点大小 = 该词在该歌评论的出现密度（log）
 - 一条曲线沿时间顺序连接所有亮起的圆点——这就是这个词"找宿主"的轨迹

- 曲线**逐段绘制**（从左到右，每段 0.4s），让用户跟着看路径"延伸"

- 屏幕下方动态文本：故事化叙述这个词的旅程

默认进入：

- t=0: 45 个灰色圆点 + 时间轴出现
- t=0.5s: 自动选中**"失眠"**作为示范
- t=0.8s: 路径开始绘制——从《认真的雪》（2006）开始，连到《动物世界》、《暧昧》、《丑八怪》、《其实》、《尘》.....
- t=4s: 路径完成，下方文字 fade-in 该词的故事

交互：

- 点击其他关键词：当前路径淡出，新路径绘制
- hover 路径上任一亮起的圆点：浮出"这首歌评论里，'失眠'出现了 N 次"

右下故事化文本示例（不同词不同文本）：

失眠 ·
"2008 年它寄居在《认真的雪》。
2014 年搬到了《丑八怪》。
2022 年又出现在《尘》。

二十年里，失眠这个词
一直在他不同的歌下面，
找不同的人。"

关键 trick：

- 一个词的路径用 SVG `<path>` + `stroke-dashoffset` 动画从 0 → 总长度，模拟"画出来"
- 路径不是直线相连，是带 quadratic curve 的弯曲连线，更像"搜寻轨迹"
- 路径颜色映射词的情绪属性

文案：

- 标题: **一种感觉，找了二十年宿主。**
- 副标题: 同一个词，在不同的歌下，找不同的人。
- 关键词候选: 失眠 · 后悔 · 长大 · 想她 · 原谅 · 一个人 · 回不去 · 再见

Scene 12 · `<DateAnchorWallScene />` NEW

意图：评论里有人写下具体日期——"2018 年 3 月 14 日"、"高三那年的冬天"、"我妈走的那天"。让这些被记住的时刻**集体显形**。

核心数据：用正则 + NER 从评论文本提取日期类表达。

结构：

```
<DateAnchorWallScene>
  |— <DateCard × 24 />      24 张日期卡，散布
  |— <CalendarSilhouette /> 背景淡淡的日历轮廓
  |— <DateDetail card={focused} />
```

视觉：

- 背景：极淡的日历网格（30% 透明度，几乎看不见）
- 屏幕上**散落 24 张日期卡**——不规则布局，每张卡片：
 - 顶部一个**手写体日期**："2018.3.14" 或 "高三那年冬天"
 - 下方一段评论摘录（30-50 字）：写下这个日期的人在评论里说了什么
 - 底部 mono 小字：歌名 · 年份
- 卡片大小不一（200-320px 宽），轻微旋转角度
- 卡片纸张质感，但比 P07 便签更小、更密

默认进入：

- t=0: 屏幕只有背景日历轮廓
- t=0.5s: 卡片**按时间顺序**从左到右、从上到下依次 fade-in（错开 80ms）——这个顺序本身就是叙事
- t=4s: 全部就位
- t=5s: 屏幕中央叠出一行字："这些日期，是某些人想被自己记住的那一天。" 停留 3s 后淡出

交互：

- hover 任一卡片：放大 1.06 + 发光
- 点击：卡片飞到中央放大，显示完整评论 + 该歌曲名 + 卡片归属年份

关键 trick：

- 卡片可以**手动排成不规则布局**（masonry / packed），让画面有"散落的笔记本残页"的感觉
- 屏幕角落一个动态计数器："N 条评论里写下了具体日期"

文案：

- 标题（屏幕左上手写体）：**被记住的那些日子。**
- 副标题: 这些不是发歌的日子。是有些人，想让自己记住的日子。
- 中央叠字: 这些日期，是某些人想被自己记住的那一天。
- 计数器: N 条评论里写下了具体日期

Scene 13 · <DustListeningScene />

（mox 原 P16 焕新——让《尘》成为情绪高潮）

结构：

<DustListeningScene>	
— <RotatingCover />	缓转封面
— <SpectrumHalo />	封面背后的频谱光环
— <FullWidthSpectrum />	屏幕底部全宽频谱
— <FloatingLyrics />	左右两侧卡拉OK高亮
— <PlaybackControls />	右上极简控制
— <ProgressTimecode />	左上极细进度条

视觉：

- 整屏黑色背景
- 屏幕中央：《尘》专辑封面，宽度 35vw，**缓慢自转**（30s/圈）
- 封面背后散出一圈**频谱光晕**——围绕封面四周用 SVG 画径向 32 根细线，每根高度跟随真实频谱数据
- 封面下方：手写体大字“尘”
- 屏幕底部铺满一条频谱（高度 100px，渐变粉到橙）
- 屏幕**左侧**和**右侧**各浮一条歌词或精选评论，卡拉OK高亮模式：
 - 当前句白色发光
 - 已过句变灰
 - 用 mox 现有的 `.lyric-line.active` 思路
- 右上角极简控制：▶ || ⏻ （点击切换播放状态）
- 左上角：极细进度条（1px 高 × 80px 宽）+ mono 时间戳 01:23 / 04:17

默认进入：

- t=0: 屏幕全黑
- t=0.3s: **音乐先响起**（dust mp3 淡入，1.5s）
- t=2s: 封面从黑里“显影”出来（filter blur 40px → 0, opacity 0 → 1, 2s）
- t=2.5s: 频谱光环和底部频谱条同时出现，开始跟乐
- t=3s: 左右歌词淡入
- t=4s: 右上控制和左上进度条淡入

关键 trick：

- **声音先于视觉**——这是这页的核心仪式感
- 封面自转是循环的，不会停
- 如果用户切到这页时音频还没加载好，就先放静态封面 + fakeSpectrum

文案：

- 没有顶部标题。这页就是**听一首歌**。
- 仅左上小字: 现在播放 · 《尘》 · 2024
- 角落: 13 / 14 DUST · NOW PLAYING

Scene 14 · <PeaceFinaleScene />

(mox 原 P17 焕新——18 万条同句汇聚)

结构：

<PeaceFinaleScene>	
├── <FloatingPeaceText />	持续生成"世界和平"
├── <ConvergeAnimation />	最终汇聚动画
├── <BigPeaceWord />	最终大字
├── <FinalCaption />	小字说明
└── <ReplayButton />	"从头再听一遍"

视觉：

- 整屏全黑
- 持续生成"世界和平"四字飘字：
 - 每条手写体（Ma Shan Zheng）
 - 不同字号（30-80px）
 - 不同米色调
 - 从屏幕外随机位置慢慢飘入
 - 飘到一个目标位置停 1.5s 后淡出
 - 每秒生成 1-2 条
 - 持续 18s

默认进入：

- t=0: 上一页音乐 fade-out 完成，屏幕全黑
- t=2s: 第一条"世界和平"出现
- t=2-18s: 持续生成，画面累积约 15-25 条同时漂浮
- t=18s: 停止生成新的飘字
- t=20s: 屏幕上剩余约 10 条"世界和平"**全部慢慢汇聚到屏幕中央**（CSS transform 缓动）
- t=22s: 这些字**叠加重合**成一个**超大号的"世界和平"**（衬线 900，font-size: clamp(80px, 14vw, 240px)，颜色金色）
- t=23s: 大字下方手写小字 fade-in：

N 万 N 千多人，
在不同的歌、不同的年里，
写下同样这四个字。

- t=24s: 再下方更小 mono:

2006 – 2026 · 45 首歌 · 网易云评论数据可视化

- t=25s: 最下方一个小按钮: "从头再听一遍" (点击回到 Scene 01)

关键 trick:

- 整页没有任何肖像图——纯黑底，让"世界和平"四字独自承担
- 飘字汇聚动画是这份作品的最后一个动作——一定要做得克制，不要花哨

文案:

- 大字: 世界和平
- 大字下手写: N 万 N 千多人，在不同的歌、不同的年里，写下同样这四个字。
- 最下方 mono: 2006 – 2026 · 45 首歌 · 网易云评论数据可视化
- 按钮: 从头再听一遍

全局节奏建议

把 14 页分成四个乐章:

乐章一·入场(轻): 01 封面 → 02 数字
乐章二·铺陈(密): 03 时间轴 → 04 潜伏期 → 05 凌晨 → 06 共用句
乐章三·深入(缓): 07 便签墙 → 08 语言搬家 → 09 长评
乐章四·情感(重): 10 情绪粒子 → 11 词宿主 → 12 日期 → 13 尘 → 14 世界和平

每个乐章之间用不同的切换动画:

- 乐章一内部: crossfade (柔和)
- 乐章二内部: 横向 slide-left (密集区给点动作)
- 乐章三内部: subtle crossfade
- 乐章四内部: fade-to-black (情感终曲)

现在请你回我

1. 数据问题: A B C D I (凌晨 / 潜伏期 / 共用句 / 日期 / 词宿主) 这五个新角度, 哪些数据你能拉到? 拉不到的就从 14 页里剔除, 我重新调整结构。
2. 优先级: 14 页全做工作量大, 要不要先做一个 MVP 5 页 (01 封面 + 02 数字 + 03 时间轴 + 13 尘 + 14 世界和平), 跑通整个工程框架, 看效果满意了再扩展?

3. **审美口味**：上面我假设了一些手写体 + 米色纸 + 黑底色这套调调（沿用 mox 偏深沉的色感）。如果你想要更"年轻"或更"实验性"（比如把字体调成更现代的 grotesque、把米色换成冷灰），告诉我，我整套调整。

回答这三个，我直接开始写代码。